

Lecture 5: Spanning Tree Routing

CS 356: Introduction to Computer Networks

Instructor: Venkat Arun

P&D Chapter 3.2

Borrows some images from “Computer Networks: A Systems Approach” by Peterson and Davie

Recap

There are three types of addresses commonly used on the internet. We build up the higher layers of addressing from the lower layers

- Ethernet addresses: Completely flat. Only guarantee* is that the addresses are unique
- IP addresses: Hierarchical address. Depends on *where* the device is on the internet
- Domain name: Human interpretable. DNS maps domain name to IP address. For performance optimization, mapping may depend on where the user is

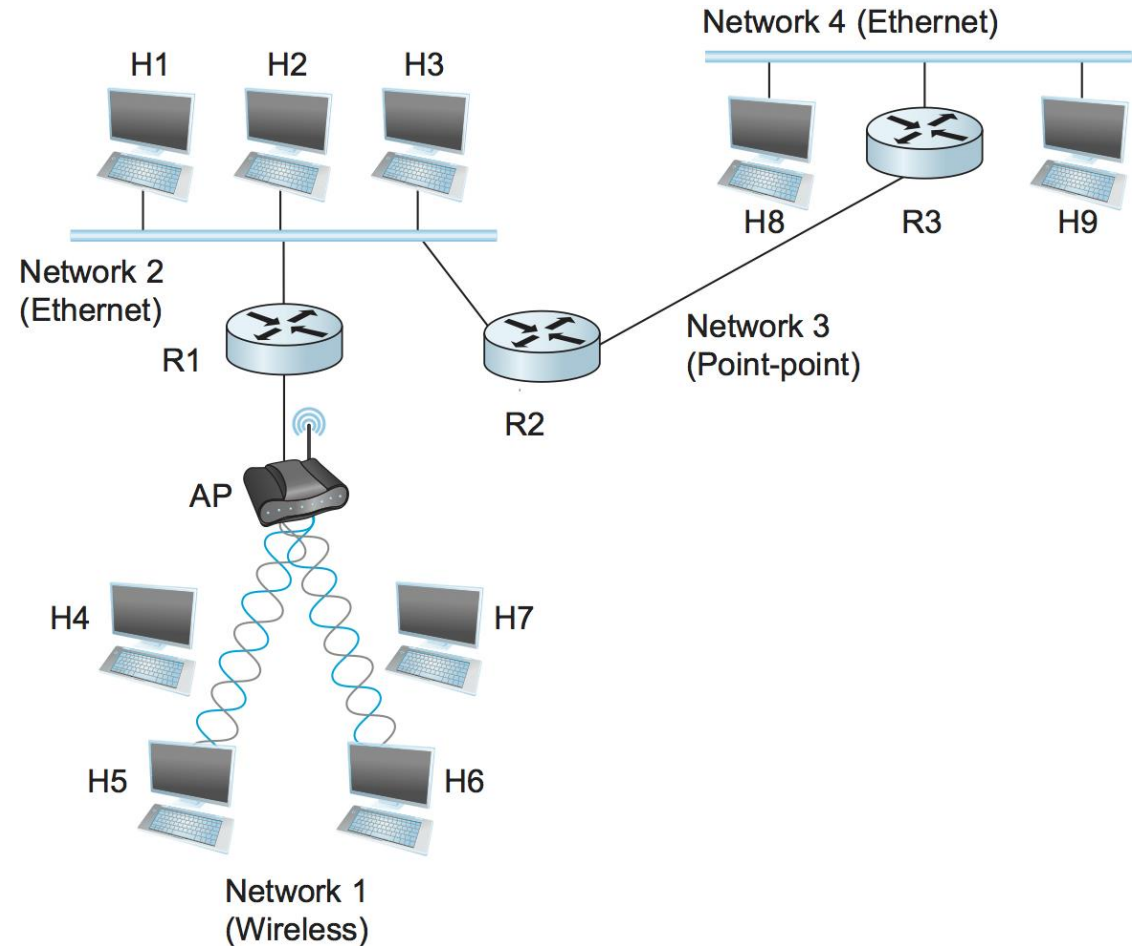
We are discussing ethernet today, which one layer below IP

Within an ethernet network, MAC addresses are used for routing (Q: do the packets also contain IP addresses?)

Routing is done according to the method we will discuss today

As far IP is concerned, all hosts connected to the ethernet network are just “Next hop”.

This is an example of an “overlay network”, a recurring pattern in networking.

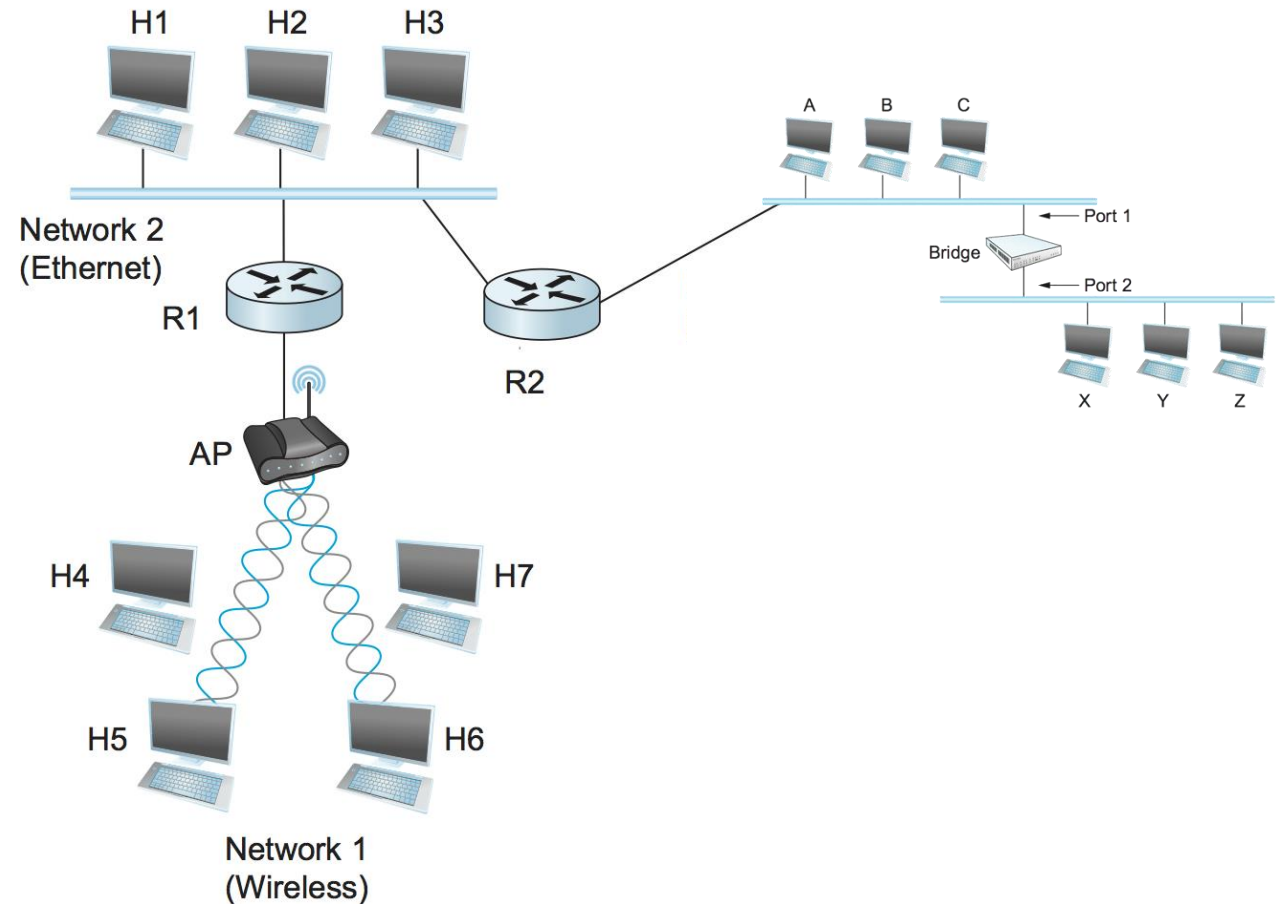


We are discussing ethernet today, which one layer below IP

The IP layer connects as yet another endpoint to the host link. Network 2 connects R1, R2, H1, H2, and H3.

Similarly Network 3 connects R2 to hosts A, B, C, X, Y and Z

The bridge only looks at ethernet headers and is unconcerned with IP



Simple ethernet bridge

```
table = {}
```

```
def pkt_rcv(pkt):
```

```
    # Update the table
```

```
    if pkt.src_addr not in table:
```

```
        # Note: Do not confuse port of the bridge with TCP/UDP port number
```

```
        table[pkt.src_addr] = pkt.src_port
```

```
    # Forward the packet
```

```
    if pkt.dst_addr in table:
```

```
        forward(table[pkt.dst_addr], pkt)
```

```
    else:
```

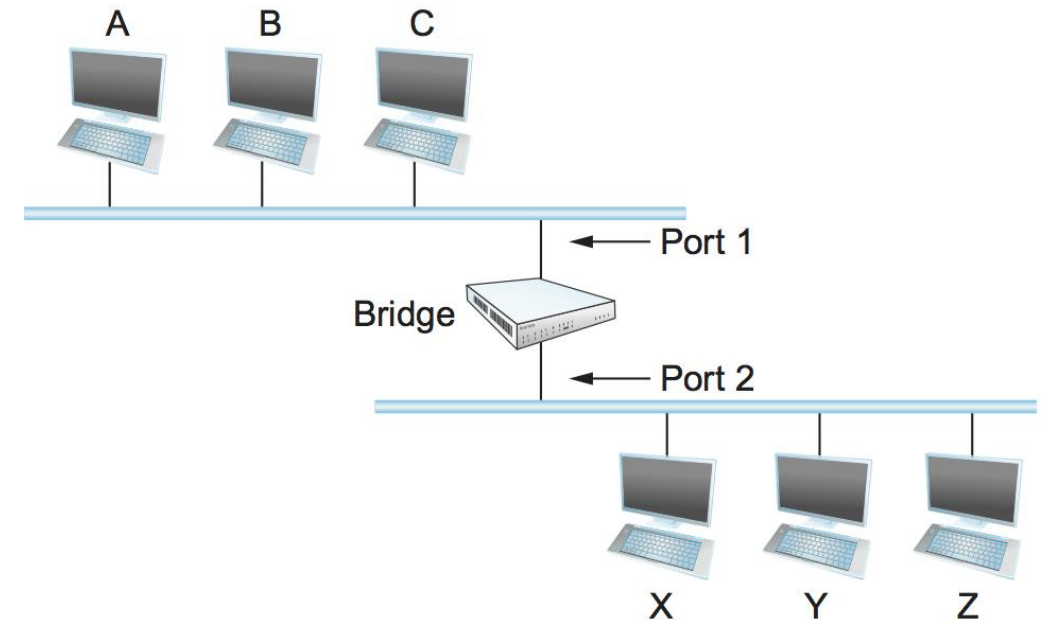
```
        for port in all_ports:
```

```
            forward(port, pkt)
```

```
def forward(port, pkt):
```

```
    if port != pkt.src_port:
```

```
        # Send pkt on port
```



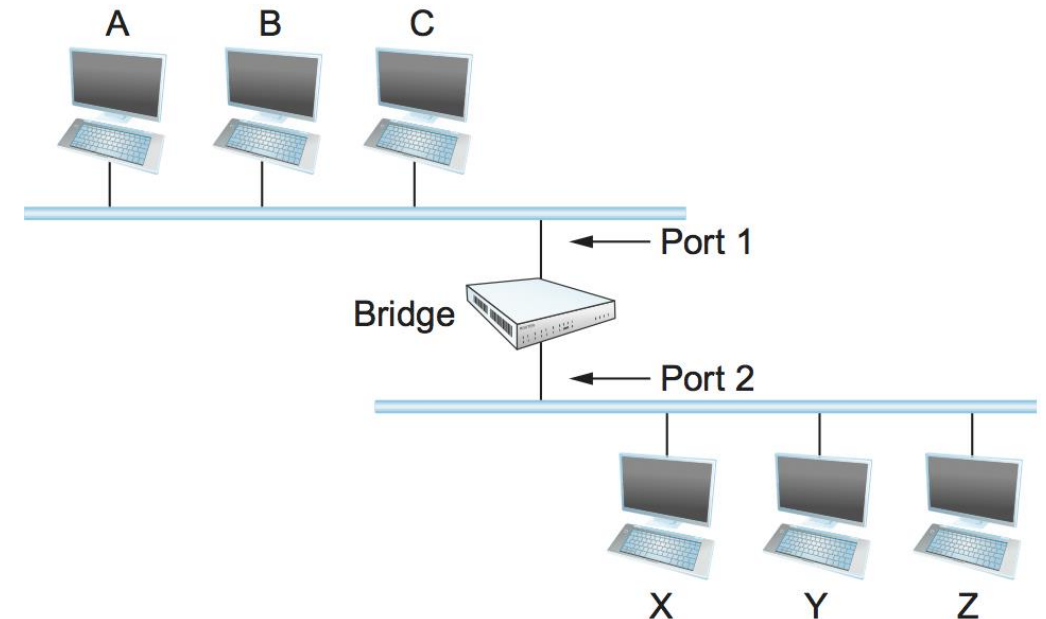
Simple ethernet bridge

The horizontal lines are physical broadcast media. What is said by one port is heard by all the others.

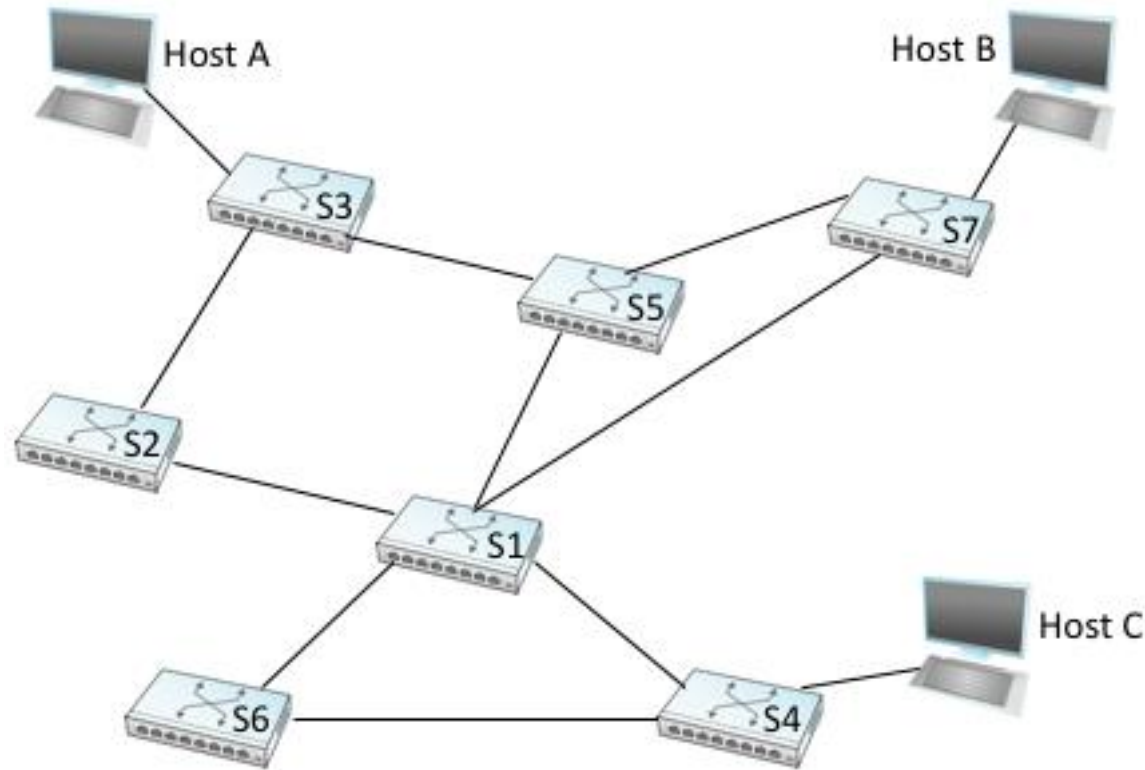
Today, wireless media are the only broadcast media. In the early days, people would connect multiple things to the same physical copper wire.

Problem: when X is talking to Y, if the bridge forwards all packets from port 2 to port 1, then those packets are needlessly heard by A, B and C. During that time, A cannot talk to B. This increases congestion

Solution: The bridge somehow learns to not forward packets between X and Y to the other side. It only forwards when necessary



This technique fails when there are cycles

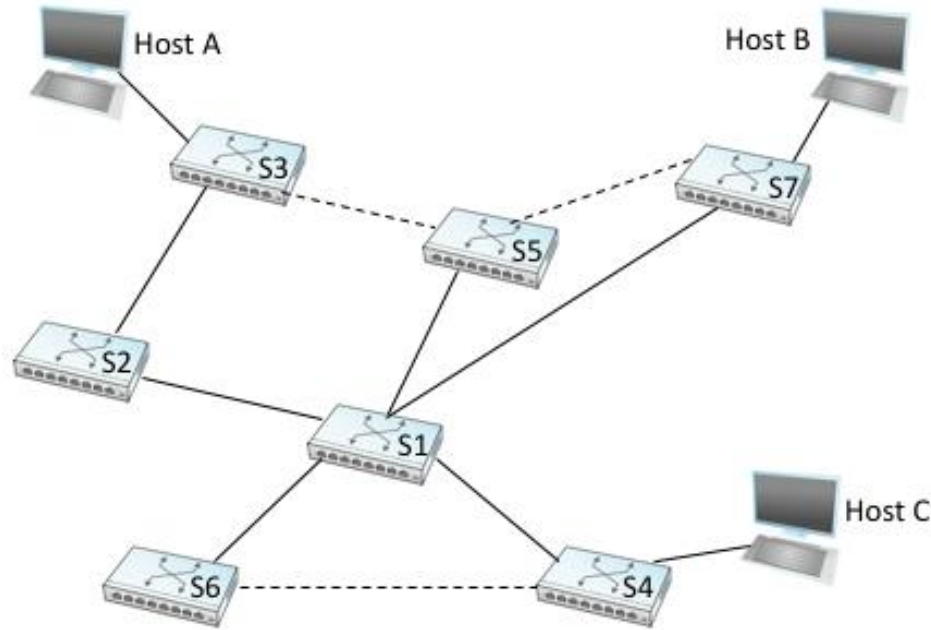


Suppose `table` is empty for all the switches. When host A sends a packet to B, the packets will circulate forever inside the network on *all* the switches because they forward it indefinitely

Quick and dirty fixes

- Remember all packets that a switch sent. Do not forward the same packet twice
 - If an end host sends identical packets, say to retransmit information, the network will refuse to forward the second one
 - Switches need a *lot* of *fast* memory
 - Q: How does this compare to updating the routing table?
- Create a “time to live (TTL)” field in the packet. Switches decrement it each time. If it reaches 0, they do not forward
 - Much lower overhead
 - Avoids infinite circulation, but packets can still circulate
 - IP headers have a TTL field, but it is used as a safety mechanism, not the first line of defense
 - Note: we are talking of ethernet here. Ethernet bridges ignore IP headers

Proper fix: switches should disable enough links that cycles no longer exist



Goal: All nodes should be connected, but there should be no cycles => tree

Q: Should the network admin just avoid adding those extra links in the first place then?

Q: How would you do this?

Three approaches

- Have a centralized server run a routing algorithm (here, spanning tree) and tell everyone which links to turn on
 - Essential idea behind Software Defined Networking (SDN)
 - Problems?
 - Single point of failure. Plus, what happens if we cannot reach this centralized server?
- Design a distributed algorithm where each switch operates independently and yet the system converges to a good answer
 - Self repairing and more resilient to failures
 - Algorithms harder to design, but are a one-time effort
 - Today, it is the fallback option when SDN fails. SDN is optional, self-repairing networks are not
- Cop out: get a human to fix issues
 - Sometimes needed even today

Distributed spanning tree algorithm

Strategy:

- All switches pick a common “root” switch

Whichever has the lowest ethernet address (assumes unique addresses)

- All other switches select the shortest path to that root switch. Any links not on the shortest path of at least one switch are removed

If two paths have the same length, pick hop with lower ethernet address

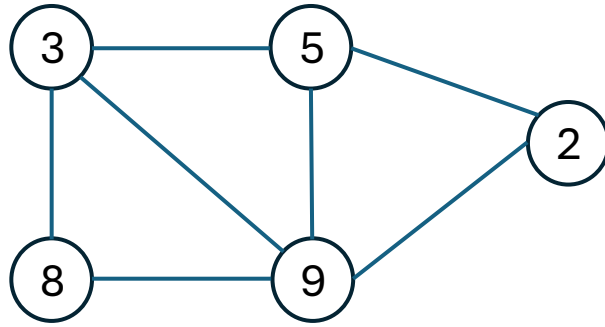
- If information is stale, timeout so that if a link fails, we pick a different tree

PS: Distributed algorithms are always hard

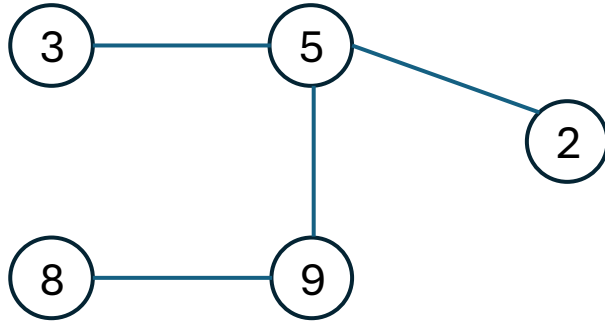
Logic at each switch

- If I think I am the root switch:
 - Tell all my neighbors that I have a path to the root (me) with 0 hops
 - If someone advertises a path to a root with address smaller than me, stop thinking that I am the root switch and start thinking they are the root switch
- If I think someone else (say X) is the root switch
 - Keep track of the shortest path to X that anyone has advertised
 - Tell all my neighbors about this shortest path to X
 - If someone else advertises a path to X that is shorter than mine (in case it is the same length, pick the one whose sender has the smaller ID), update my idea of a shortest path. Re-advertise

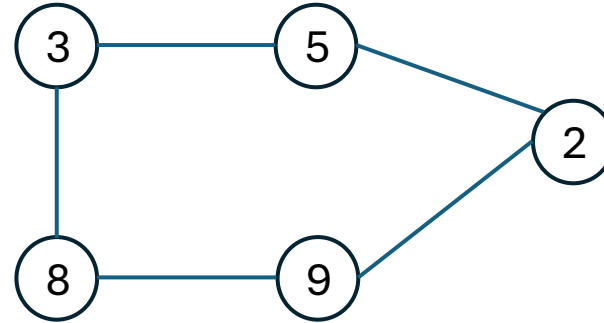
Question: what does this topology converge to?



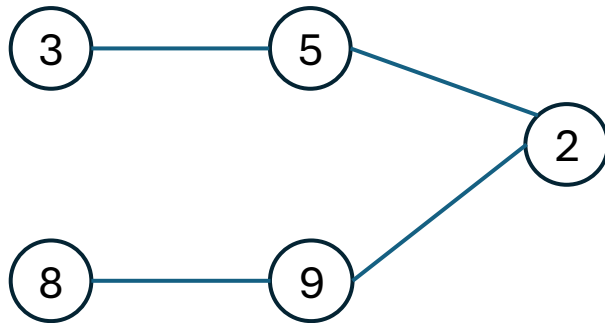
A



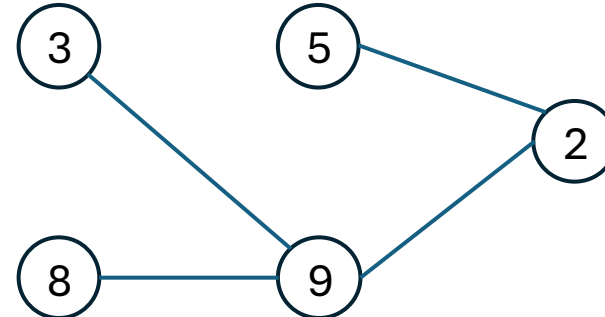
B



C



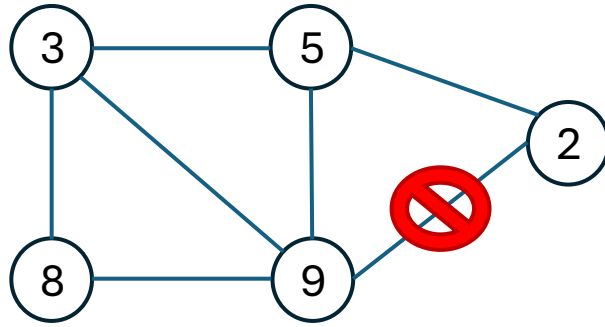
D



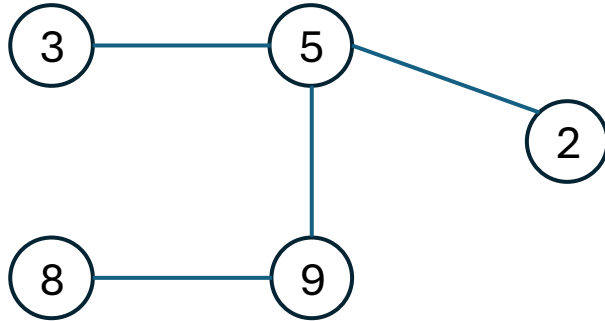
Logic at each switch: handling failures

- If I think I am the root switch:
 - Tell all my neighbors that I have a path to the root (me) with 0 hops
 - If someone advertises a path to a root with address smaller than me, stop thinking that I am the root switch and start thinking they are the root switch
 - Periodically advertise myself to my neighbors to assure them I'm "alive"
- If I think someone else (say X) is the root switch
 - Keep track of the shortest path to X that anyone has advertised
 - Tell all my neighbors about this shortest path to X
 - If someone else advertises a path to X that is shorter than mine, update my idea of a shortest path. Re-advertise
 - Upon receiving a periodic message, create corresponding update to other neighbors
 - If I haven't heard from the root in a while, start thinking that I'm root again

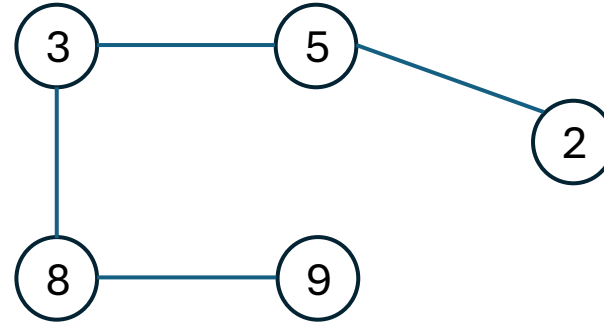
Question: what does this topology converge to?



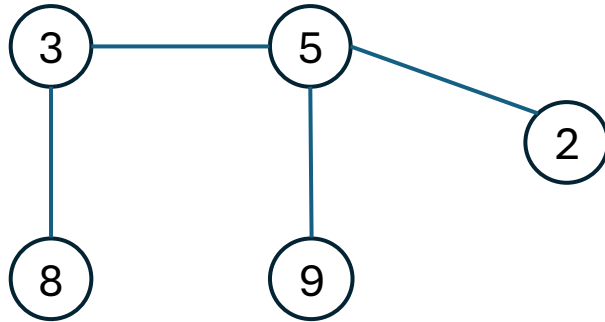
A



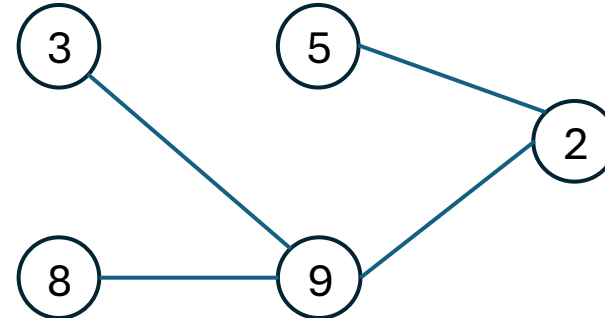
B



C



D



Let us look at the original algorithm again:

Strategy:

- All switches pick a common “root” switch

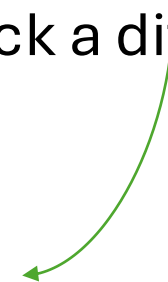
Whichever has the lowest ethernet address (assumes unique addresses)

- All other switches select the shortest path to that root switch. Any links not on the shortest path of at least one switch are removed

If two paths have the same length, pick hop with lower ethernet address

- If information is stale, timeout so that if a link fails, we pick a different tree

Is this rule needed to ensure loop freeness?



Distributed spanning tree algorithm

- Is this guaranteed to produce a tree?
- Will the tree be “optimal”? What does “optimal” mean?
- Does the best routing scheme always lead to a tree?

Recall: the bridge algorithm

```
table = {}
```

```
def pkt_rcv(pkt):
```

```
    # Update the table
```

```
    if pkt.src_addr not in table:
```

```
        # Note: Do not confuse port of the bridge with TCP/UDP port number
```

```
        table[pkt.src_addr] = pkt.src_port
```

```
    # Forward the packet
```

```
    if pkt.dst_addr in table:
```

```
        forward(table[pkt.dst_addr], pkt)
```

```
    else:
```

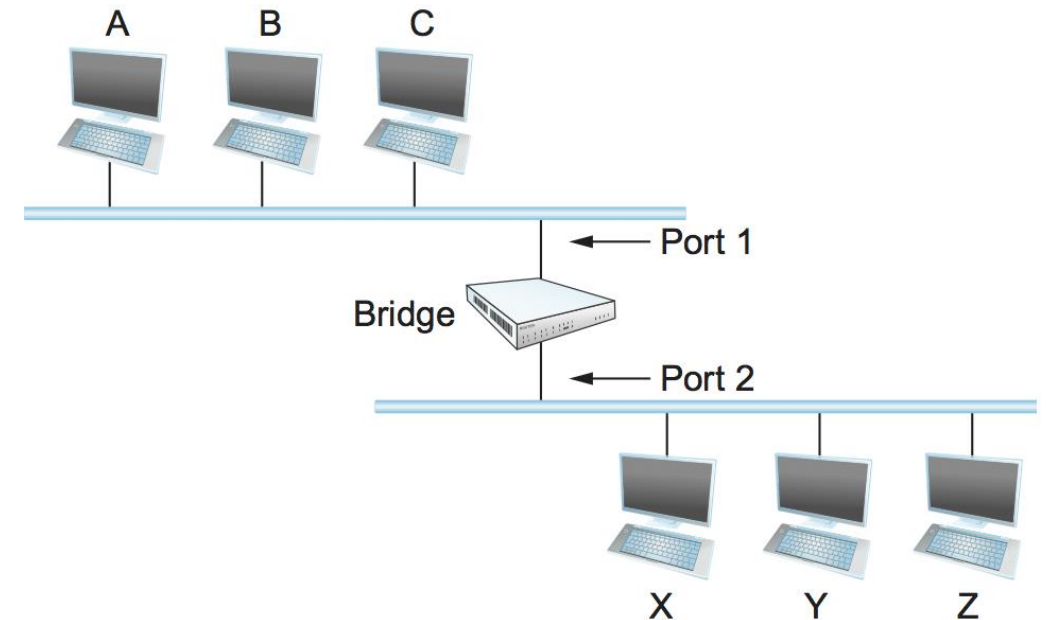
```
        for port in all_ports:
```

```
            forward(port, pkt)
```

```
def forward(port, pkt):
```

```
    if port != pkt.src_port:
```

```
        # Send pkt on port
```



Interaction between spanning tree and the bridge algorithm

- The spanning tree changes.
 - Temporarily, it can have cycles before convergence
 - Even if there are no cycles, the tree changes in response to failures
- What the bridge algorithm learned in that time may be incorrect.
- How would you fix it?

Interaction between spanning tree and the bridge algorithm

```
table = {}
```

```
def pkt_rcv(pkt):
```

```
    # Update the table
```

```
    if pkt.src_addr not in table:
```

```
        table[pkt.src_addr] = pkt.src_port
```

```
    # Forward the packet
```

```
    if pkt.dst_addr in table and table[pkt.dst_addr] is active:
```

```
        forward(table[pkt.dst_addr], pkt)
```

```
    else:
```

```
        for port in all_ports:
```

```
            forward(port, pkt)
```

```
def forward(port, pkt):
```

```
    if port != pkt.src_port and port is active:
```

```
        # Send pkt on port
```